TechPost ELEVATED

DC – AI – Routing – Switching – Security – Wireless – Automation

DNS64 and NAT64 on SRX Series

An HPE Juniper Networking Techpost

Steven Jacques

2025-02-12

Contents

1 DNS64 and NAT64 on SRX Series 3

1.1 Introduction . . . 3
1.2 How DNS64 Works . . . 3
1.3 Configuring NAT64 on SRX . . . 4
1.4 Verifying the NAT 64 Configuration on SRX . . . 5
1.5 Useful links . . . 6
1.6 Glossary . . . 6
1.7 Acknowledgments . . . 6

List of Figures

1 NAT64 with DNS64 processing flow 4
2 Lab topology 4

1 DNS64 and NAT64 on SRX Series

Steven Jacques - 02/12/2025

In this short post, we'll look at configuring the SRX for 6-to-4 NAT (NAT64) when using IPv6-only clients with an external DNS64 server. We'll also quickly examine how the mechanism to dynamically perform this translation works.

1.1 Introduction

If you're reading this blog, you probably know the score with IPv6. Actual statistics on the use of IPv4 versus IPv6 vary, but a good indicator is the relative access to Google services which can be seen here. Whatever the true breakdown is of v4 to v6 across different internet services and applications, it's fair to say that at the moment, v4 is still the primary method used, and the pace of change towards IPv6 is relatively slow. Many internet sites and services don't have IPv6 access enabled.

This presents a challenge for businesses and operators wanting to roll out IPv6 (which they may want to do for various reasons, not least to increase IP address capacity for cloud scale, IoT device provisioning, and more). If an IPv6-only endpoint inside an organization, or connected to an operator network, wants to access an internet service that is pure IPv4, well… that's a problem.

Fortunately, there is a simple solution - NAT64, combined with DNS64.

1.2 How DNS64 Works

When the v6-only endpoint wants to connect to a v4-only website, we have to perform translation of the v6 source and destination addresses to become a v4 packet. How do we do this? We can't possibly do 1-to-1 mappings for every service and site on the internet - this has to be dynamic. DNS64, working in conjunction with a NAT64 translator such as Juniper's SRX platform, solves this problem. A quick mention for our SRX NAT64 options - we have a small form factor 1RU appliance (the SRX4600 and upcoming SRX4700), a modular and scalable chassis-based system (SRX5000 series), and a scale-out architecture providing practically unlimited capacity and scale - this is Connected Security Distributed Services Architecture, or CSDS (the datasheet for which can be found here).

DNS64 servers (there are various ones publicly available, including from Google and Cloudflare) use a *well-known NAT64 prefix* in their DNS response. So, the logical process is as follows:

1. Client sends a v6 DNS request to resolve the site it is attempting to reach. This goes out to the DNS64 server using IPv6;
2. Server responds with the resolved address. It uses the well-known /96 prefix 64:ff9b::/96 (defined in RFC6052) as the first 96 bits of the resolved address. The remaining 32 bits are the **IPv4 address** of the target website (which the server knows….it's a DNS server, after all). Essentially, this means that the DNS64 server is synthesising a v6 AAAA record from the v4 A record;
3. The client connects to the resolved IPv6 address. It is unaware that this is anything other than the genuine address of the target site;
4. The NAT64 gateway (SRX) recognises the well-known prefix as configured and performs destination IP NAT on the received packet by extracting the last 32 bits and using those as the destination IPv4 address in the translated packet;

5. The SRX also performs standard source translation, by whatever method is configured - PAT, Port Block Allocation, or something else - to change the v6 source to a public v4 source;
6. Client packet is forwarded in IPv4 form to the v4 target site.

The process is represented graphically below:

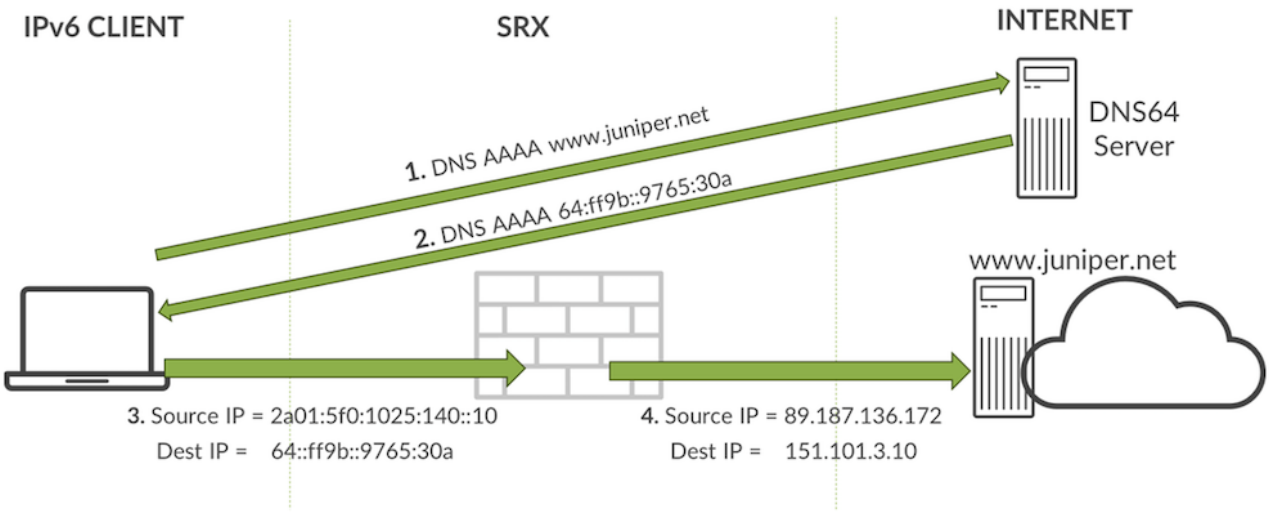


Figure 1: NAT64 with DNS64 processing flow

1.3 Configuring NAT64 on SRX

For this, we're using the simplest of labs, but let's show it as a reference and include the used IP addresses:

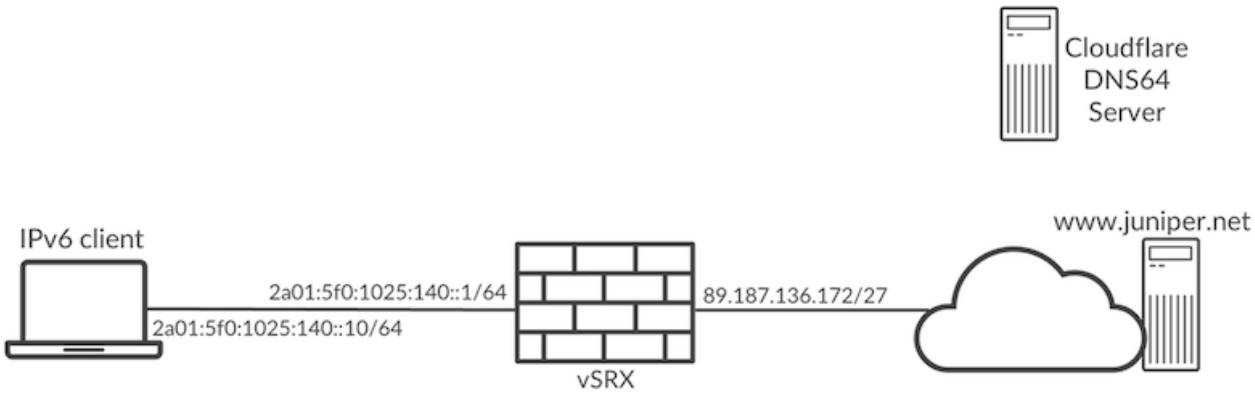


Figure 2: Lab topology

We won't cover setting up interfaces, zones, security policies and so on here - let's get straight to the relevant NAT portion. The destination NAT simply looks for the well-known NAT64 prefix and then uses the 'inet' keyword to translate to the IPv4 address, which is extracted from the last 32 bits of the IPv6 packet. Note that this is a static NAT, not a destination NAT (in terms of SRX configuration hierarchy).

```
root@vsrx-p4# show security nat static
rule-set nat64 {
   from zone trust;
   rule nat64 {
      match {
         source-address ::/0;
         destination-address 64:ff9b::/96;
```

```
        }
        then {
            static-nat {
                inet;
            }
        }
    }
}
```

For the source NAT, we're just going to translate to the external interface for this. In our lab, it's using a public IP, so that interface address can reach the internet, as would be possible on a NAT64 gateway. Any source NAT configuration is viable - we could just as easily have a public address pool, we could have used Port Block Allocation - the fact that the source address match is looking for any IPv6 address doesn't matter.

Here's a small gotcha - note that the criteria for the source NAT uses a destination address which is 'any IPv4'. This is because static NAT is evaluated first, so the destination address will already have been translated to a v4 address. We have to include this condition, because the v6 client might want to access some native v6 services - we don't want the source NAT occurring in that case, and this condition would prevent a match there.

```
root@vsrx-p4# show security nat source
rule-set untrust {
    from zone trust;
    to zone untrust;
    rule untrust {
        match {
            source-address ::/0;
            destination-address 0.0.0.0/0;
        }
        then {
            source-nat {
                interface;
            }
        }
    }
}
```

And… that's it. That's all there is to it. Our IPv6 client is set up and configured to use Cloudflare's DNS64 servers, so let's give it a try.

1.4 Verifying the NAT 64 Configuration on SRX

We'll use undoubtedly the best site on the entire internet for this test - www.juniper.net. Firstly, let's check the name resolution:

```
root@vsrx-pod-04:~# host www.juniper.net
www.juniper.net is an alias for www-aem.junipercloud.net.
www-aem.junipercloud.net is an alias for cdn.adobeaemcloud.com.
cdn.adobeaemcloud.com is an alias for adobe-aem.map.fastly.net.
adobe-aem.map.fastly.net has address 151.101.3.10
adobe-aem.map.fastly.net has address 151.101.67.10
adobe-aem.map.fastly.net has address 151.101.131.10
adobe-aem.map.fastly.net has address 151.101.195.10
adobe-aem.map.fastly.net has IPv6 address 64:ff9b::9765:30a
adobe-aem.map.fastly.net has IPv6 address 64:ff9b::9765:430a
adobe-aem.map.fastly.net has IPv6 address 64:ff9b::9765:830a
adobe-aem.map.fastly.net has IPv6 address 64:ff9b::9765:c30a
```

Here we can see the well-known NAT64 prefix in use for the v6 versions of the resolved addresses. For the first one, we can see 9765:30a in the last 32 bits, which in decimal is 151.101.3.10, mapping correctly to the v4 address seen above it.

So, let's try a ping and see if works:

```
root@vsrx-pod-04:~# ping www.juniper.net
PING www.juniper.net(64:ff9b::9765:30a (64:ff9b::9765:30a)) 56 data bytes
64 bytes from 64:ff9b::9765:30a (64:ff9b::9765:30a): icmp_seq=1 ttl=59 time=7.03 ms
```

Looks good. Note that the SRX will also extract the IPv4 destination and perform NAT64 correctly if we ping/connect to some resource using mixed or embedded notation (i.e. - in this case - if we ping 64:ff9b::151.101.3.10).

Finally, let's check that the SRX is processing and translating the traffic in the way we'd expect. We'll run a CuRL from our IPv6 client to the Juniper site, and then check the SRX session table:

```
root@vsrx-p4> show security flow session destination-port 443
Session ID: 8590086418, Policy name: permit/4, Timeout: 2, Session State: Valid
  In: 2a01:5f0:1025:140::10/38380 -> 64:ff9b::9765:30a/443;tcp, Conn Tag: 0x0, If: ge-0/0/0.0,
      Pkts: 14, Bytes: 1558,
  Out: 151.101.3.10/443 -> 89.187.136.172/13998;tcp, Conn Tag: 0x0, If: ge-0/0/2.0, Pkts: 15,
      Bytes: 7444,
Total sessions: 1
```

Here, we can see that the translation is working correctly - the destination IP address is being translated from v6 to v4 using the last 32 bits of the v6 address, and the source v6 address is being translated to the SRX v4 interface IP. All good. That's a wrap!

1.5 Useful links

- Junos OS NAT guide https://www.juniper.net/documentation/us/en/software/junos/nat/nat.pdf
- Pathfinder - for full list of supported NAT features https://apps.juniper.net/home/segment?segment=Security&subSegment=Security

1.6 Glossary

- CSDS: Connected Security Distributed Services
- DNS: Domain Name System
- IPv4: Internet Protocol, version 4
- IPv6: Internet Protocol, version 6
- NAT: Network Address Translation
- NAT64: Network Address Translation, IPv6 to IPv4
- v4/V4: Shorthand for IPv4
- v6/V6: Shorthand for IPv6

1.7 Acknowledgments

Karel Hendrych for sanity checking, corrections, and the use of his wonderful, IPv6-enabled lab.



DC – AI – Routing – Switching – Security – Wireless – Automation

© Copyright 2026 Hewlett Packard Enterprise Development LP. The information contained herein is subject to change without notice. The only warranties for Hewlett Packard Enterprise products and services are set forth in the express warranty statements accompanying such products and services. Nothing herein should be construed as constituting an additional warranty.
Hewlett Packard Enterprise shall not be liable for technical or editorial errors or omissions contained herein.
Trademark acknowledgments, if needed. All third-party marks are the property of their respective owners.

HEWLETT PACKARD ENTERPRISE

HPE.com



